

AuditSure

A Neuro-Symbolic GST Compliance Verification System for Indian MSMEs

B.Tech Capstone Project - Technical Report

Architecture: OWL 2 Ontology (Layer 1) → Python + Z3 Symbolic Reasoner (Layer 2) → LLM Explanation Bridge (Layer 3)

Ontology v1.1.0 • 184 automated tests passing • 0 OWL inconsistencies (HermiT-verified)

Table of Contents

Abstract

1. Introduction and Motivation

2. System Architecture

3. Layer 1 - Legal Knowledge Ontology

4. Layer 2 - Symbolic Reasoning Engine

5. Layer 3 - LLM Explanation Bridge

6. Testing and Validation Methodology

7. Demonstration Interface

8. Limitations

9. Future Extensions

10. Conclusion

References

Appendix A: Module Inventory

Appendix B: Sample Proof Object Output

Abstract

AuditSure is a neuro-symbolic compliance verification system that determines an Indian MSME's obligations under the Central and Integrated Goods and Services Tax Acts without asking a large language model to perform any legal reasoning. All legal determinations - registration liability, composition eligibility, input tax credit eligibility, e-invoicing and e-way bill applicability, return obligations, interest, penalty, refund, and appeal calculations - are made by a Z3 symbolic constraint solver operating over facts pulled dynamically from a validated OWL 2 ontology encoding of GST law. A large language model is used only in the final stage, to convert an already-verified, citation-backed Proof Object into a plain-English explanation for a non-technical business owner; it is explicitly forbidden from introducing any legal claim not already present in that Proof Object.

This report documents the complete system: a 175-class, 348-individual OWL 2 ontology built from the CGST Act, IGST Act, and their Rules (Sections extracted and cross-checked against a January 2026 GST Manual, and independently verified consistent by the HerMiT description-logic reasoner); a ten-phase Python/Z3 reasoning engine covering six compliance domains; a deterministic, testable LLM-prompt bridge; and a working command-line demonstration interface. The system is validated by 184 automated tests, including boundary-exact unit tests for every statutory threshold and an integration suite that deliberately probes for - and in two cases found and fixed - genuine cross-module legal errors that unit testing alone had not caught.

1. Introduction and Motivation

Indian MSMEs face a GST compliance regime that is procedurally intricate: registration thresholds vary by State category, the composition scheme carries several independent disqualifying conditions, input tax credit is subject to five simultaneous statutory conditions plus two separate time-bars, and financial consequences (interest, penalty, refund limitation, appeal pre-deposit) each follow their own computation rules. A single incorrect judgment - claiming ITC one condition short of eligible, or missing a Section 24 compulsory-registration trigger because turnover alone looked fine - carries real financial and legal exposure.

A purely LLM-based compliance assistant is unsuitable for this domain: language models do not guarantee logical consistency, cannot be exhaustively boundary-tested the way a program can, and may quietly conflate similarly-worded provisions (as this project's own testing repeatedly demonstrated is easy to do even for a carefully-written symbolic engine - see Section 8). AuditSure's central design decision is therefore to separate the system into three layers with a hard boundary on what each is allowed to do:

- Layer 1 (Ontology) is the single source of truth for what the law says: every threshold, rate, and citation lives here and nowhere else.
- Layer 2 (Z3 Reasoner) is the single place where legal logic is evaluated: it reads Layer 1's facts and a business's own data, and produces a machine-checked verdict with a full citation trail.
- Layer 3 (LLM) is presentation only: it may rephrase and organize Layer 2's output, but is contractually (via its system prompt) forbidden from adding any new legal content.

This separation is what makes the system's answers auditable: every verdict traces back to a specific ontology individual and a specific line of Python/Z3 logic, rather than to an opaque model weight.

2. System Architecture

The end-to-end data flow is a strict pipeline: a BusinessProfile (the facts about one MSME - turnover, State, registration status, invoices, supplies, prior tax payments, refund claims, appeals) is passed through six Z3-backed compliance engines in sequence, each appending its findings to a single shared Proof Object, which is finally handed to the LLM bridge for explanation.

AuditSure: Neuro-Symbolic GST Compliance Architecture

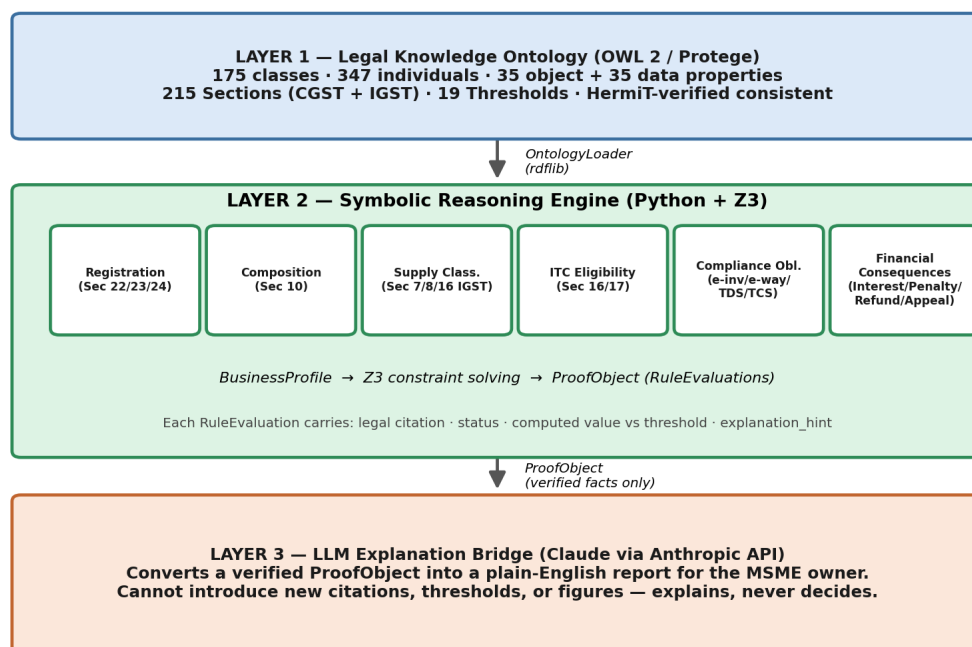


Figure 1. The three-layer neuro-symbolic architecture.

Two structural properties of this design matter beyond the obvious layering:

2.1 The ontology is consulted at runtime, not compiled into the code

Every engine calls OntologyLoader methods (`get_threshold`, `get_section`, `is_special_category_state`, `get_business_type_names`, ...) rather than hard-coding a rupee figure or a State name. A future amendment - the GST Council raising the composition threshold, for instance - is a one-line edit to the ontology's Turtle file; no Python file needs to change, and the change is automatically picked up everywhere that threshold is used.

2.2 Later phases consume earlier phases' verdicts, not just raw facts

Phase 6's interest and penalty engines do not re-derive which input tax credit was wrongly availed - they read Phase 4's already-computed VIOLATED findings out of the Proof Object built so far. This is a deliberate anti-drift measure: the legal judgment of 'is this ITC wrongly availed' is made in exactly one place in the codebase.

Cross-Phase Consumption: Interest & Penalty Read Prior Verdicts, Don't Re-Derive Them

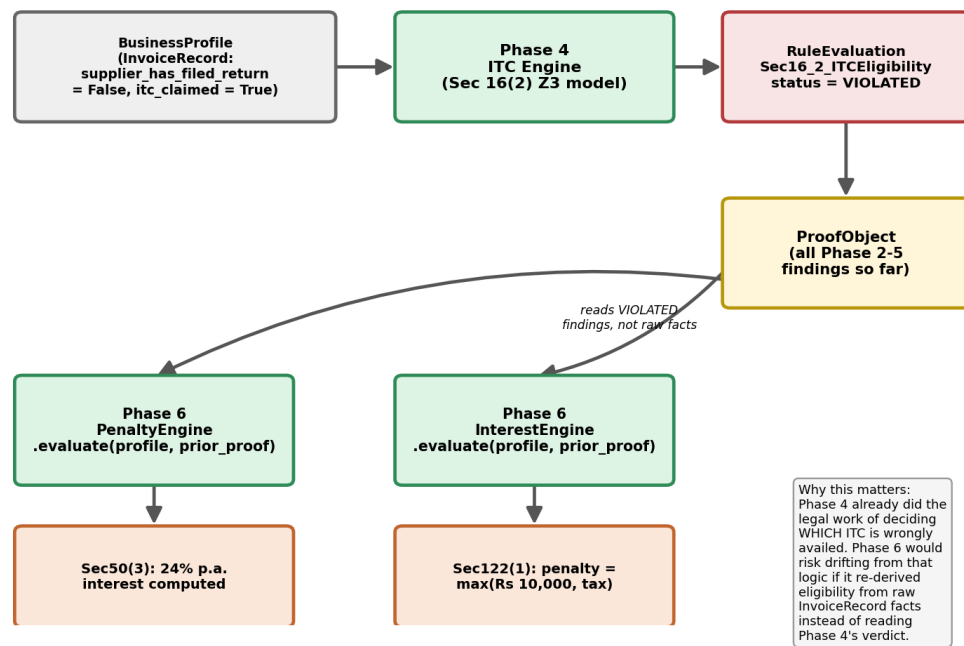


Figure 2. Interest and penalty calculations read Phase 4's verdicts rather than re-implementing Section 16(2) eligibility logic a second time.

3. Layer 1 - Legal Knowledge Ontology

The ontology (auditsure_gst_ontology.ttl, OWL 2 / Turtle, also exported to RDF/XML for direct Protege import) was built from the CGST Act 2017, IGST Act 2017, their Rules, and the GST (Compensation to States) Act, using a January 2026 GST Manual as the source for section text, thresholds, and notification citations. It is organized into 19 modules (Core, Business, Registration, Taxation, Supply, Input Tax Credit, Invoice, Returns, Payment, Refund, Assessment, Audit, Demands and Recovery, Penalty, Advance Ruling, Appeals, Transitional, Legal References, and Thresholds), tagged with a `gst:module` annotation for programmatic filtering.

Metric	Count
Named classes	175
Named individuals	348
Object properties	35
Data properties	35

Sections modelled (CGST + IGST)	215, across 30 chapters, 3 Acts
Threshold individuals	20
States (including 3 dual-typed Union Territories)	31
Restriction axioms	13
equivalentClass axioms	4
AllDisjointClasses blocks	8

3.1 Validation methodology

The ontology was validated at three levels, each catching different classes of error:

- Syntax: parsed with rdflib on every edit; zero tolerance for parse failures before proceeding.
- Structural QC: a Python script checks for properties missing a domain/range, cyclic subclass hierarchies, duplicate property names, and CamelCase/lowerCamelCase naming-convention violations. All checks currently pass with zero violations.
- Logical consistency: the full ontology is loaded into Hermit (a complete OWL 2 DL reasoner) via owlready2 after every substantive edit. Two real inconsistencies were found and fixed this way during development - both caused by an owl:inverseOf property whose declared range forced an unintended class membership on unrelated individuals (Rules/Schedule documents being forced into the disjoint Chapter class). The ontology has been consistent (0 inconsistent classes) since both fixes.

3.2 Honesty about scope: documented placeholders instead of invented figures

Per the project's governing rule - never assert a legal fact not verified in the source material - several commonly-known figures were deliberately left unmodelled where they were not found in the extracted manual text:

- HSN-wise GST rate slabs (5% / 12% / 18% / 28% etc.) are not present in the source manual (a Bare Acts/Rules compilation, not a tariff notification), so only the statutory rate cap (Section 9(1)'s 20% CGST ceiling) is asserted.
- The full Article 279A(4)(g) constitutional list of special-category States is not reproduced in the manual; only the Section 22 explanation's express exclusions (Jammu & Kashmir and six named States) are asserted directly from source text.
- The QRMP scheme's turnover eligibility ceiling is referenced by notification number only, not by verified figure, so return-frequency choice is left as an informational note rather than a hard threshold.

Each such placeholder carries a `gst:sourceNote` annotation explaining exactly what is and is not verified, so a future maintainer (or Layer 3, if it were ever asked) cannot mistake an unverified note for an authoritative figure.

4. Layer 2 - Symbolic Reasoning Engine

Layer 2 was built in nine phases, each producing a working, independently-tested increment before the next began. The phase boundaries follow compliance domains (registration, then composition, then supply classification, and so on) rather than arbitrary code modules, so that each phase maps onto a self-contained question an MSME owner would actually ask.

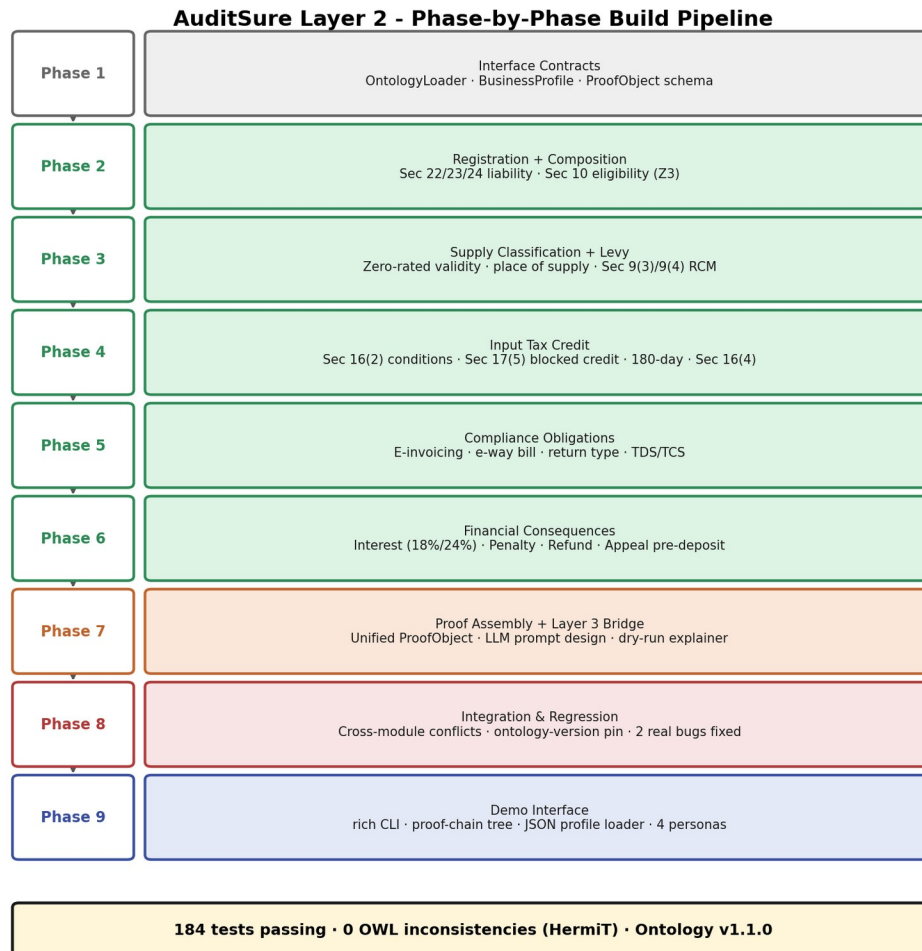


Figure 3. The nine build phases, in dependency order.

4.1 Why Z3, not if/else

Several provisions interact with a specific legal priority that is easy to encode incorrectly as an ordered chain of Python conditionals: Section 24 compulsory registration overrides both the Section 22 turnover threshold and a Section 23 exemption; Section 17(5) blocked credit is an absolute bar independent of - and checked before - Section 16(2) eligibility. Encoding each such rule as a single Z3 boolean formula, and reading both the final verdict and the human-readable 'why' back out of the same solved model, removes the risk of the priority being implemented backwards in one branch while looking correct in isolated testing. Z3 is also used for genuine satisfiability queries beyond simple evaluation - for example,

`solve_minimum_turnover_for_exemption()` in the registration engine asks Z3 what turnover level would change a business's registration outcome, rather than looking up a pre-computed answer.

4.2 Module summary

Phase	Module(s)	Core legal basis
2	registration_engine.py, composition_engine.py	Sec 22/23/24 (registration); Sec 10 (composition levy)
3	supply_classification_engine.py, levy_mechanism_engine.py	Sec 7/8, Sec 16 IGST (supply classification); Sec 9(3)/9(4) (reverse charge)
4	itc_engine.py	Sec 16(2) (five ITC conditions); Sec 17(5) (blocked credit); Sec 16(4) (time bar); Rule 37 (180-day rule); Sec 10(4) (composition ITC bar)
5	einvoice_ewaybill_engine.py, return_obligation_engine.py, tds_tcs_engine.py	Rule 48(4) (e-invoicing); Rule 138 (e-way bill); Sec 37/39/44 (returns); Sec 51/52 (TDS/TCS)
6	interest_engine.py, penalty_engine.py, refund_engine.py, appeal_engine.py	Sec 50 (interest); Sec 122 (penalty); Sec 54 (refund); Sec 107 (appeal pre-deposit)
7	explanation_prompt.py, layer3_explainer.py, auditsure_pipeline.py	Proof Object assembly; LLM prompt design; top-level facade

4.3 Bugs found and fixed during development

Three genuine defects were identified and corrected during Layer 2's construction, each through deliberate cross-checking rather than by accident:

- A composition taxpayer's inward invoices were evaluated against ordinary Section 16(2) eligibility instead of the absolute Section 10(4) bar on any input tax credit - found while probing for cross-module interactions in Phase 8, before a single new test was written.
- E-invoicing applicability did not consider composition status - a composition taxpayer issues a Bill of Supply, not a tax invoice, so Rule 48(4) cannot apply to it regardless of turnover.
- Proof Object metadata carried a hard-coded ontology version string ("1.0.0") that never actually reflected the loaded ontology file, so a version bump would have silently gone unrecorded in every generated report.

All three are now permanent regression tests in `test_phase8_integration.py`.

5. Layer 3 - LLM Explanation Bridge

Layer 3's only input is a completed Proof Object; its only output is Markdown-formatted prose. The bridge is split into two modules specifically so that prompt construction can be tested without any network access or API credentials:

- `explanation_prompt.py` is pure and deterministic: the same Proof Object always produces the same prompt string, grouped by status in the priority order VIOLATED → INDETERMINATE → SATISFIED → NOT_APPLICABLE, so an LLM reading top-down naturally foregrounds what matters most.

- `layer3_explainer.py` performs the actual Anthropic API call, and auto-activates a dry-run mode (returning the exact prompt instead of a live response) whenever no `ANTHROPIC_API_KEY` is configured. This is not a stub - it is complete integration code exercised end-to-end in this project's own development environment, which itself has no API key.

The system prompt's central guarantee is stated explicitly and tested (`test_system_prompt_forbids_introducing_new_legal_content`): the model may only use facts, figures, and citations already present in the Proof Object text it is given, and must never soften, omit, or contradict a VIOLATED finding. This is the concrete mechanism by which the neuro-symbolic split bounds the LLM's hallucination surface for legal content to zero - all legal reasoning already happened in Z3.

6. Testing and Validation Methodology

The system carries 184 automated tests (183 executed plus 1 environment-gated skip for a live API call), organized in two tiers:

Test file	Tests	Scope
<code>test_phase2.py</code>	21	Registration + composition, boundary-exact on every threshold
<code>test_phase3.py</code>	18	Supply classification, place-of-supply consistency, reverse charge
<code>test_phase4.py</code>	27	ITC eligibility, blocked credit, 180-day rule, time bar
<code>test_phase5.py</code>	23	E-invoicing, e-way bill, return obligations, TDS/TCS
<code>test_phase6.py</code>	29	Interest, penalty, refund, appeal pre-deposit
<code>test_phase7.py</code>	19 (1 skipped)	Prompt determinism, dry-run explainer, pipeline facade
<code>test_phase8_integration.py</code>	22	Cross-module conflicts, ontology-version pin, realistic personas
<code>test_phase9_cli.py</code>	26	JSON profile loading, CLI subprocess smoke tests

6.1 Boundary-exact unit testing

Every statutory threshold is tested at the exact boundary value, not merely “above” and “below” it, because the Act's own comparison operators are inconsistent across provisions and an off-by-one error is the most common way to misencode a threshold: Section 22 registration liability uses a strict ‘exceeds’ comparison (turnover exactly at Rs 20 lakh is NOT yet liable), while Section 10 composition eligibility uses an inclusive ceiling (turnover exactly at Rs 1.5 crore IS still eligible). A dedicated test guards specifically against copy-pasting one convention into the other's engine.

6.2 Integration testing: looking for conflicts, not just confirming correctness

Phase 8's suite is deliberately different in kind from the per-phase unit tests. Rather than re-confirming that each engine is individually correct, it probes realistic combinations - a business simultaneously

eligible for composition and making inter-State supply; a Section 23-exempt agriculturist who also has a blocked-credit ITC claim; the same special-category-state determination checked for agreement between the registration and composition engines across seven States - specifically to surface interaction bugs that isolated unit tests cannot see. Both real bugs described in Section 4.3 were found this way, before the corresponding regression tests were written.

6.3 Ontology-version pinning

A dedicated test (`test_ontology_version_is_pinned`) asserts the loaded ontology's `owl:versionInfo` against a hard-coded expected string, and a companion test pins the twelve most load-bearing threshold values directly. If the ontology is edited without a deliberate version bump and human review of this pin, the test suite fails loudly rather than silently continuing to pass against a changed legal fact.

6.4 Determinism

`test_same_profile_produces_identical_verdicts_across_runs` confirms that running the identical `BusinessProfile` through the full pipeline twice produces byte-identical rule IDs and statuses. This is the concrete, testable difference between a symbolic reasoning engine and an LLM-only system: the former is reproducible by construction.

7. Demonstration Interface

A command-line interface (`cli.py`, built on the rich library) provides three ways to exercise the system:

```
python3 cli.py --demo growing_manufacturer      # built-in example personas
python3 cli.py --profile my_business.json       # a user-supplied JSON profile
python3 cli.py --interactive                    # guided prompts, no JSON needed
```

Output includes a colour-coded summary table, a proof-chain tree (every Section/Rule that fired, grouped by compliance module and sorted with violations first), an “Action Required” panel, and the Layer 3 explanation. A JSON profile loader (`profile_loader.py`) performs the string-to-Decimal/date/enum conversions needed to deserialize a `BusinessProfile` from a plain JSON file - infrastructure equally usable by a future REST API. Four example personas are included, one specifically constructed to demonstrate the cross-module registration/composition conflict described in Section 6.2.

8. Limitations

The following limitations are deliberate scope boundaries, documented in the relevant module's docstring at the point they matter, rather than gaps discovered after the fact:

- Numeric GST rate slabs (5%/12%/18%/28%) are not modelled; composite/mixed supply rate determination (Sec 8(a)/(b)) is confirmed at the classification level but left INDETERMINATE on the actual rate.
- Sector-specific exemptions from e-invoicing (SEZ units, insurers, banks, GTAs, government departments) and from e-way bills are not modelled - only the turnover/value threshold check is performed.

- Section 9(3)/9(4) reverse-charge determination takes the notified-category status as an input flag rather than deriving it from an exhaustive notified-goods/services list, since that list lives in separate rate notifications outside the source manual's scope.
- Section 51 TDS threshold checking is performed per-invoice, not per-contract; a contract artificially split across multiple sub-threshold invoices would not be caught.
- QRMP quarterly-filing eligibility is noted informationally without a hard numeric threshold, since that specific figure was not verified against the source manual text during ontology construction.
- The system evaluates a business's stated facts; it does not verify those facts (e.g. it trusts `payment_made_to_supplier` as given rather than checking a bank statement).
- No live LLM call has been executed against Layer 3 in this development environment (no API key configured) - the dry-run path is fully tested, but actual LLM output quality/tone is unverified beyond the deterministic prompt it receives.

9. Future Extensions

The core classes deliberately reused across the ontology - `LegalPerson`, `GovernmentAuthority`, `LegalReference` (Act/Section/Rule/Notification/Circular), and `Threshold` - were designed to generalize beyond GST law, per the original project brief's request for cross-domain extensibility:

- MSME Development Act, 2006: the ontology already carries a stub `gst:MSME` class with a `sourceNote` explaining that MSME classification (investment/turnover-based Micro/Small/Medium tiers) is a separate statute; a new module would add MSMEAct-specific `Threshold` and `Section` individuals reusing the existing `Business/BusinessType` hierarchy.
- Income Tax Act: presumptive taxation schemes (Sec 44AD/44ADA) and TDS provisions parallel GST's own TDS mechanism closely enough that the existing `PaymentObligation` and `Threshold` classes would transfer with minimal new modelling.
- Labour Codes / EPFO / ESIC: employee-count and wage thresholds for statutory registration follow the same "Threshold triggers RegistrationCategory" pattern already built for Sec 22/24, suggesting the Registration module's Z3 formula structure (not just its data) could be reused directly.
- Companies Act, 2013: filing obligations (annual returns, board report thresholds) mirror the `Return/ReturnFrequency` pattern from Phase 5.

In each case, the recommended integration path is the one this project itself followed: extract and cross-check source statutory text into a versioned OWL module, validate with `HermiT` before writing any Python, then build one Z3 engine per compliance question with boundary-exact tests before integrating.

10. Conclusion

AuditSure demonstrates that a legally-grounded compliance system can be built without asking a language model to be the source of legal truth. Its ontology is the sole repository of legal facts, independently validated for logical consistency; its Z3 engines make every determination through machine-checked, boundary-tested constraint solving with a full citation trail back to the ontology; and its LLM layer is architecturally prevented from introducing new legal content, however fluent its prose.

The 184-test suite - and in particular the two real cross-module bugs it caught before they could reach a user - is offered as evidence that this architecture's value is not merely conceptual: building it this way found and fixed mistakes that a less rigorously structured system would likely have shipped.

References

Garg, Rakesh (Adv., CA) and Garg, Sandeep (CMA, FCA). GST Manual, Sixth Half-Yearly Edition, 01 January 2026. (Source for CGST Act 2017, CGST Rules 2017, IGST Act 2017, IGST Rules 2017, GST (Compensation to States) Act 2017 and Rules, GST Appellate Tribunal Rules 2023, and GST Appellate Tribunal (Procedure) Rules 2025 text, section numbering, thresholds, and notification/circular citations used throughout Layer 1.)

Central Goods and Services Tax Act, 2017 (No. 12 of 2017), Government of India.

Integrated Goods and Services Tax Act, 2017 (No. 13 of 2017), Government of India.

W3C OWL 2 Web Ontology Language, Document Overview (Second Edition), 2012.

Glimm, B., Horrocks, I., Motik, B., Stoilos, G., Wang, Z. Hermit: An OWL 2 Reasoner. *Journal of Automated Reasoning*, 2014.

de Moura, L., Bjørner, N. Z3: An Efficient SMT Solver. *TACAS* 2008.

Appendix A: Module Inventory

File	Purpose
auditsure_gst_ontology.ttl / .owl	Layer 1: the complete OWL 2 ontology (Turtle and RDF/XML)
ontology_loader.py	Typed Python accessors over the ontology (Phase 1)
business_profile.py	Input schema: BusinessProfile and all nested record types (Phase 1, extended each phase)
proof_object.py	Output schema: ProofObject and RuleEvaluation (Phase 1)
registration_engine.py / composition_engine.py	Phase 2 engines
supply_classification_engine.py / levy_mechanism_engine.py	Phase 3 engines
itc_engine.py	Phase 4 engine
invoice_ewaybill_engine.py / return_obligation_engine.py / tds_tcs_engine.py	Phase 5 engines
interest_engine.py / penalty_engine.py / refund_engine.py / appeal_engine.py	Phase 6 engines
phase2_pipeline.py ... phase6_pipeline.py	Sequential pipeline wiring, one per phase
explanation_prompt.py / layer3_explainer.py	Phase 7: Layer 3 bridge
auditsure_pipeline.py	Phase 7: stable top-level facade (run_compliance_check, generate_explanation)
cli.py / profile_loader.py	Phase 9: demo CLI and JSON profile deserializer
examples/*.json	Phase 9: four demonstration business personas
test_phase2.py ... test_phase9_cli.py	184 automated tests across all phases

Appendix B: Sample Proof Object Output

Abbreviated JSON output for a single RuleEvaluation, showing the structure every module produces:

```
{
  "rule_id": "Sec22_RegistrationLiability",
  "module": "Registration",
  "status": "VIOLATED",
  "legal_citation": "Sec 22(1) proviso, CGST Act 2017",
  "section_number": "22",
  "computed_value": 2500000.0,
  "threshold_value": 2000000.0,
  "threshold_id": "ThresholdRegistrationGeneral20L",
  "explanation_hint": "Sharma Textiles's aggregate turnover of Rs
    2,500,000 exceeds the Rs 2,000,000 threshold under Sec 22(1),
    so registration is mandatory."
}
```